VISUALIZING HIGH-DIMENSIONAL DATA

t-SNE

t-Distributed Stochastic Neighbor Embedding

A Seminar on Nonlinear Dimensionality Reduction for Visualization

OVERVIEW

Agenda

1. Why reduce dimensions?
2. What is t-SNE?
3. How t-SNE works
4. The crowding problem
5. Tuning perplexity
6. Case study: CIFAR-10
7. t-SNE for word vectors
8. Limitations & alternatives

MOTIVATION

Why Reduce Dimensions?

- **Simplify without losing too much information**

  Dimensionality reduction condenses a dataset while preserving the structure that matters.

- **Merge correlated features — “feature extraction”**

  E.g. a car’s mileage and age are both signs of wear, so an algorithm can fold them into one feature.

- **Faster training, less memory and disk**

  Feeding a supervised learner fewer, denser features speeds it up — and can occasionally boost performance.

- **t-SNE takes this further — for human eyes**

  It reduces data purely so we can plot and see structure that would otherwise live in hundreds of dimensions.



DEFINITION

What Is t-SNE?

t-SNE (t-Distributed Stochastic Neighbor Embedding) is a nonlinear technique that reduces dimensionality while trying to keep similar instances close together and dissimilar instances far apart.

1

Nonlinear

Unlike PCA, it doesn’t just find a flat rotation of the data.

2

Visualization-first

Built to make 2D / 3D plots humans can read, not for general preprocessing.

3

Preserves local structure

Neighborhoods matter most; global distances are not the priority.

Developed by Laurens van der Maaten & Geoffrey Hinton (2008) — used to visualize MNIST digits, word-vector spaces, gene-expression profiles, and more.

MECHANISM

How t-SNE Works

1

Similarity in High-D

For every pair of points, compute the probability that one would pick the other as a neighbor, using a Gaussian centered on each point. Nearby points score high; far points score near zero.

2

Similarity in Low-D

Randomly place points in 2D/3D, then define similarities there too — but with a heavy-tailed Student-t distribution instead of a Gaussian.

3

Optimize

Use gradient descent to move the low-D points so their similarity pattern matches the high-D one as closely as possible — minimizing KL divergence.

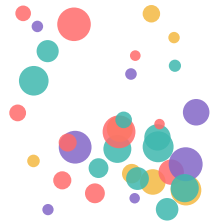
The result: two probability distributions — one from the original space, one from the map — that the algorithm tries to align.

KEY INSIGHT

The Crowding Problem — Why the “t”?

- In high-dimensional space, a point can have many neighbors that are all roughly equidistant — there’s room for that.
- Squeezed into just 2D, there isn’t enough room: those neighbors would have to crowd on top of each other near the center.
- A heavy-tailed Student-t distribution for low-D similarities lets moderately distant points repel more strongly, so clusters spread out instead of collapsing.
- This fix is exactly what separates t-SNE from the original SNE algorithm.

Without heavy tails: crowded



With Student-t: spread out



PARAMETER TUNING

The Perplexity Parameter

Perplexity roughly sets how many effective nearest neighbors each point considers when building the high-D similarity distribution. Typical range: 5–50.

L

Low (~5)

Focuses on very local structure. Can fragment natural clusters into scattered pieces.

M

Medium (~30)

A balanced, commonly-used default — usually the best starting point.

H

High (50+)

Considers broader neighborhoods. Can blur fine-grained structure together.

Tip: always try a few different perplexity values — and remember, cluster shapes can look different on every run.

CASE STUDY

Semantic Clusters Emerge Automatically

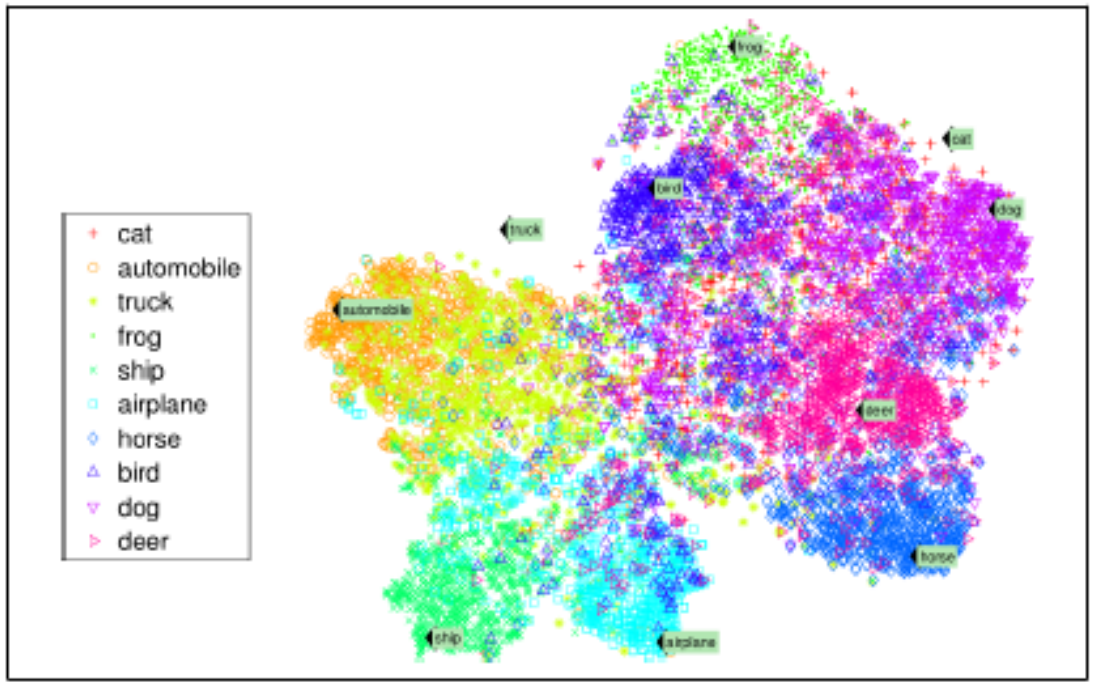


Figure 1-9. Example of a t-SNE visualization highlighting semantic clusters[3]

t-SNE visualization of CIFAR-10 image classes. Reproduced with permission from Socher, Ganjoo, Manning & Ng (2013), via Géron, Hands-On Machine Learning.

- Vehicles (car, truck, airplane, ship) cluster apart from animals (cat, dog, horse, deer, bird, frog).
- Horses sit closer to deer than to birds — reflecting genuine semantic similarity, not just pixel similarity.
- Reminder: cluster sizes and the empty space between clusters aren't meaningful — only local neighborhoods are preserved.

APPLIED EXAMPLE

t-SNE for Visualizing Word Vectors

```
from sklearn.manifold import TSNE

tsne = TSNE(n_components=2, n_iter=1000)
X_2d = tsne.fit_transform(
    model.wv[model.wv.vocab]
)

coords_df = pd.DataFrame(
    X_2d, columns=['x', 'y']
)
coords_df['token'] = \
    model.wv.vocab.keys()

_ = coords_df.plot.scatter(
    'x', 'y', figsize=(12, 12),
    marker='.', s=10, alpha=0.2
)
```

n_components

Dimensions of the output map — 2 for a 2D scatterplot, 3 for a 3D one.

n_iter

Number of optimization iterations, like epochs in neural-network training. More iterations can help, with diminishing returns.

Result

Each word gets an (x, y) coordinate to plot — words used in similar contexts land near each other.

The same pattern works for any embedding — images, genes, users, and more.

READ WITH CAUTION

Limitations & Things to Watch For

Distances aren’t global

Cluster size and gaps between clusters don’t reliably mean anything — only local neighborhoods are preserved.

Stochastic

Results depend on the random seed; rerunning can produce a visibly different layout.

No easy out-of-sample mapping

There’s no simple formula to place a brand-new point — the algorithm typically must be rerun.

Perplexity-sensitive

Different perplexity values can tell different stories about the same data — always check a few.

Slower at scale

Naive t-SNE scales poorly with dataset size (Barnes-Hut and other approximations help).

Modern alternative

UMAP — often faster and better at preserving some global structure.

SUMMARY

Key Takeaways

- ✓ t-SNE turns high-dimensional data into a 2D/3D map you can actually look at.
- ✓ It preserves local neighborhoods by matching similarity distributions — Gaussian in high-D, Student-t in low-D.
- ✓ The heavy-tailed t-distribution solves the “crowding problem.”
- ✓ Perplexity controls the effective neighborhood size — try several values.
- ✓ Great for exploration; read distances and cluster sizes with caution.
- ✓ Works on any vector representation: images, word embeddings, gene expression, and more.

SOURCES

References & Further Viewing

- **van der Maaten, L., & Hinton, G. (2008)**

  Visualizing Data using t-SNE. Journal of Machine Learning Research, 9, 2579–2605.

- **Géron, A. — Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow**

  Chapter 1, Figure 1-9, image originally from Socher, Ganjoo, Manning & Ng (2013).

- **Lane, H., Howard, C., & Hapke, H. — Natural Language Processing in Action**

  Chapter 11, word-vector visualization example (TSNE on a Project Gutenberg corpus).

- **StatQuest with Josh Starmer**

  “t-SNE, Clearly Explained” — YouTube.